CESI
ÉCOLE D'INGÉNIEURS

# Livrable 2 – Systèmes embarqués :

# Worldwide weather watcher

**19/10/2024**




**Groupe N°4 (salle 111) :**

RINN Clément
LEMBLE Lorenzo
THOMAS Paul

# Sommaire :

Contexte du projet ..... 3

Déroulement du projet ..... 3

Présentation l'architecture du programme utilisé par la carte..... 3

Conclusion ..... 12

# Contexte du projet :

Notre startup THS a pour projet de développer un prototype de station météo embarquée pour les navires, dans le but de prévenir les catastrophes naturelles comme les cyclones. Notre station devra recueillir des données environnementales via divers capteurs (température, pression, hygrométrie, luminosité, GPS, ...) et stocke ces informations sur une carte SD pour une utilisation en temps réel et une analyse ultérieure. Notre système reposera sur un microcontrôleur Arduino (AVR ATmega328) et devra être conçu pour être simple à utiliser par les équipages maritimes, avec une documentation technique à l'appui.

# Déroulement du projet :

Le projet se déroule en 4 étapes majeures :

- L'analyse du système qui consiste à concevoir des diagrammes UML/SysML et des schémas pour décrire les fonctionnalités de la station météo.
- L'architecture du programme, dans laquelle nous allons présenter la structure générale du programme utilisée par la carte.
- La maquette, qui sera une démonstration des fonctionnalités de la carte avec un programme complet, un script de compilation/téléversement et un montage des composants.
- Et finalement la documentation qui sera une fiche technique expliquant le fonctionnement global et une documentation utilisateur pour le superviseur final.

Dans ce second livrable, nous allons nous concentrer sur l'architecture du programme, en passant en revue la structure générale de celui-ci qui sera utilisé sur la carte, avec l'utilisation des différentes variables, et fonctions.

# Présentation du pseudocode du programme utilisé par la carte :

## 1) Initialisation des variables

Pour commencer, nous allons définir les variables que l'on souhaite utiliser partout dans le programme. Il faut bien les sélectionner, mais surtout les optimiser pour ne pas consommer trop de mémoire. C'est donc pour cela que nous utilisons des structures et des énumérations. Elle vont nous permettre d'y accéder aisément, partout dans le programme.

Par exemple, la structure Parametre va gérer les différentes valeurs de la configuration (valeur actuelle, minimum, maximum, ...). La structure Config va stocker les différents paramètres de configuration et la structure Bouton va contenir les états, et informations des boutons.

```
Définir la structure Parameter avec :
    - value
    - minValue
    - maxValue

Définir la structure Config avec :
    - Structure Parameter : logInterval
    - Structure Parameter : fileMaxSize
    - Structure Parameter : timeout
    - Structure Parameter : lumin
    - Structure Parameter : luminLow
    - Structure Parameter : luminHigh
    - Structure Parameter : tempAir
    - Structure Parameter : minTempAir
    - Structure Parameter : maxTempAir
    - Structure Parameter : hygr
    - Structure Parameter : hygrMinT
    - Structure Parameter : hygrMaxT
    - Structure Parameter : pressure
    - Structure Parameter : pressureMin
    - Structure Parameter : pressureMax

Définir la structure pour les Boutons avec :
    - state : état actuel du bouton (pressé ou non)
    - pressed5s : état indiquant si le bouton est maintenu pendant 5 secondes
    - pressStart : moment où le bouton a été pressé
    - pin : numéro de la broche associée au bouton

Définir une énumération pour les Modes avec les valeurs :
    - MODE_STANDARD
    - MODE_ECONOMIQUE
    - MODE_MAINTENANCE
    - MODE_CONFIG

Définir une énumération pour la couleur de la LED avec les couleurs :
    - Rouge
    - Vert
    - Bleu
    - Jaune
    - Orange
    - Eteint
```

On attribue également au début de notre programme un pin aux boutons et à la led, on va initialiser l'horloge, et déclarer les paramètres de configuration.

```
Créer un objet Button Rouge sur le pin 2
Créer un objet Button Vert sur le pin 3
Créer un objet LED sur le pin 7
Déclarer une variable pour le Mode actuel
Déclarer une variable pour la configuration
Créer un objet clock RTC
Initialiser un chronomètre initial à la valeur actuelle de millis(), utilisé pour les intervalles des mesures
```

## 2) Fonction Setup

Au sein de la fonction setup, qui se lance au démarrage du système, nous allons chercher à initialiser l'ensemble des éléments centraux du programme tel que la LED ou l'horloge en plus de configurer les boutons.

C'est également cette fonction qui aura pour rôle d'appeler le mode configuration ou le mode standard au lancement du programme.

```
Fonction setup() :
    Lire les données EEPROM et les stocker dans la variable configuration
    Initialiser la communication série à 9600
    Initialiser l'horloge RTC
    Initialiser la LED
    Configurer redButton.pin en mode INPUT_PULLUP
    Configurer greenButton.pin en mode INPUT_PULLUP

    Appeler la fonction demarrage()

    Si le bouton rouge est pressé au démarrage :
        Appeler la fonction mettreModeConfiguration()

    Appliquer setModeStandard avec led et currentMode
```

## 3) Fonction Loop

Dans la fonction loop, on cherche à récupérer l'état des boutons en permanence. Ainsi selon l'état du bouton rouge ou vert, différents modes vont être appelés. La fonction va aussi avoir pour rôle d'appeler les fonctions pour récupérer et sauvegarder les données.

```
Fonction loop :
    Appeler la fonction etatBouton(boutonRouge) pour mettre à jour les états
    Appeler la fonction etatBouton(boutonVert) pour mettre à jour les états

    Si le bouton rouge est maintenu pressé pendant 5 secondes :
        Si currentMode est MODE_MAINTENANCE :
            Appel fonction mettreModePrecedent()
        Sinon :
            Appel fonction mettreModeMaintenance()

    Si le bouton vert est maintenu pressé pendant 5 secondes :
        Si currentMode est MODE_ECONOMIQUE:
            Appel fonction mettreModeStandard()
        Sinon si currentMode est MODE_STANDARD:
            Appel fonction mettreModeEconomique()


    donnees = acquerirDonnees()

    Si le mode actuel est MODE_MAINTENANCE :
        Afficher donnees via Serial.print
    Sinon :
        Appel fonction sauvegarderDonnees(donnees)
```

## 4) Fonction pour la couleur de la LED

Nous avons décidé de créer une fonction utilitaire, permettant de définir beaucoup plus simplement la couleur de la LED avec l'énumération crée au début du programme. Grâce à cela, nous pouvons changer la couleur de la LED beaucoup plus facilement dans le reste du programme sans se soucier du code RGB.

```
fonction mettreCouleurLed(Couleur):
    Si couleur = Rouge:
        Mettre la led à 255, 0, 0
    Sinon si couleur = Vert
        Mettre la led à 0, 255, 0
    Sinon si couleur = Bleu
        Mettre la led à 0, 0, 255
    Sinon si couleur = Jaune
        Mettre la led à 255, 255, 0
    Sinon si couleur = Orange
        Mettre la led à 255, 165, 0
    Sinon
        Mettre la led à 0, 0, 0
```

## 5) Fonction pour obtenir l'état d'un bouton

Cette fonction va avoir pour objectif de vérifier l'appui des boutons par l'utilisateur, ainsi que le temps de pression de ces derniers. Elle va sauvegarder l'état du bouton dans sa structure, en utilisant la variable fournie du bouton en paramètre. Cela permettra par la suite d'obtenir facilement l'état de n'importe quel bouton.

```
fonction etatBouton(bouton) :
    Si le bouton est appuyé :
        Si la variable état du bouton est à 0 :
            Mettre la variable de l'état du bouton à 1
            Mettre la variable de la première pression du bouton au temps actuel
        Sinon si temps actuel - variable de la première pression du bouton > 5 secondes :
            Mettre la variable de l'état du bouton pressé 5 secondes à 1
    Sinon :
        Mettre la variable de l'état du bouton à 0
        Mettre la variable de l'état du bouton pressé 5 secondes à 0
```

## 6) Fonction de démarrage

La fonction de démarrage permet de faire clignoter la LED pour signaler le démarrage, et détecte si le bouton rouge est pressé pendant ce temps avec une méthode non bloquante. Si c'est le cas, la fonction renvoie Vrai pour que cette dernière soit traitée dans la fonction setup() et ainsi démarrer le mode configuration.

```
fonction demarrage():
    Déclarer une variable locale millisPrecedent
    Déclarer une variable locale couleur à Faux. (pour le clignotement)
    Déclarer une variable nombreCycle à 0

    Tant que nombreCycle < 9 : (5 clignotements)
        Si temps actuel - millisPrecedent >= 500 :
            Mettre millisPrecedent à millis()

            Si couleur = Faux :
                Appeler fonction mettreCouleurLed(Eteint)
            Sinon :
                Appeler fonction mettreCouleurLed(Vert)
            Mettre couleur à l'inverse de couleur (!couleur)
            Incrémenter nombreCycle de 1

            Appeler fonction etatBouton(rouge)
            Si bouton rouge appuyé :
                retourner Vrai
    Fin Tant que
    retourner Faux
```

## 7) Fonctions de changement de modes

Ces fonctions ont chacune pour objectif de modifier la couleur de la LED et la valeur de la variable modeActuel. On y retrouve les fonctions associés au mode standard, économique et maintenance.

Dans le cas du mode maintenance on change également la variable modePrecedent, qui nous permettra de savoir ou revenir après avoir quitté le mode maintenance grâce à la fonction mettreModePrecedent().

```
fonction mettreModeStandard():
  Appeler fonction mettreCouleurLed(Vert)
  Mettre la variable modeActuel à MODE_STANDARD

fonction mettreModeEconomique():
  Appeler fonction mettreCouleurLed(Bleu)
  Mettre la variable modeActuel à MODE_ECONOMIQUE

fonction mettreModeMaintenance():
  Appeler fonction mettreCouleurLed(Orange)
  Mettre la variable modePrecedent à modeActuel
  Mettre la variable modeActuel à MODE_MAINTENANCE

fonction mettreModePrecedent():
  Si modePrecedent = MODE_STANDARD :
    Appel fonction mettreModeStandard()
  Sinon :
    Appel fonction mettreModeEconomique()
```

Ces fonctions vont être utilisées pour changer les différents modes en fonction de l'utilisateur, mais aussi de changer le système de sauvegarde ou d'acquisition.

## 8) Mode configuration

Le mode configuration est la partie la plus complexe du système. Elle est démarrée lorsque l'utilisateur clique sur le bouton rouge lors du démarrage du système (voir la fonction setup()). Ce mode permet à l'utilisateur de reconfigurer tous les paramètres liés au World Weather Watcher, des paramètres des capteurs aux paramètres de l'horloge. Voici comment elle est décomposée :

La fonction mettreModeConfiguration permet de prendre en charge l'entrée de l'utilisateur, et d'effectuer les commandes les plus basiques (afficher la version, réinitialiser la configuration, mettre l'heure, etc ...). Lorsque l'utilisateur cherche à modifier un paramètre du système la fonction va faire appel à une autre fonction : appliquerCommande().

```
fonction mettreModeConfiguration()
    On déclare une variable debutTemps à temps actuel
    Appeler fonction mettreCouleurLed(Jaune)
    Tant que temps actuel - debutTemps < 30 secondes
        Si l'utilisateur a tapé dans le moniteur série :
            debutTemps = temps actuel
            entree = recuperer l'entree de l'utilisateur
            Si entree == "RESET" :
                Mettre dans l'EEPROM les valeurs par défaut
            Sinon si entree == "VERSION" :
                Afficher la version du programme
            Sinon :
                Diviser l'entree en deux parties : commande et valeur
                Si commande == "CLOCK" :
                    Diviser la valeur en trois parties : heure, minute, seconde
                    Mettre l'heure, la minute et la seconde dans l'horloge
                Sinon si commande == "DATE" :
                    Diviser la valeur en trois parties : jour, mois, annee
                    Mettre le jour, le mois et l'annee dans l'horloge
                Sinon si commande == "DAY" :
                    Mettre le jour de la semaine dans l'horloge
                Sinon :
                    Appeler fonction appliquerCommande(commande, valeur)
```

La fonction appliquerCommande() permet de vérifier que la modification de l'utilisateur est bien comprise dans l'intervalle. Si c'est le cas, elle va modifier cette valeur dans la configuration, et sinon, renvoyer une erreur.

```
fonction appliquerCommande(commande, valeur) :
    si valeur >= commande.valeurMin et valeur <= commande.valeurMax :
        commande.valeur = valeur
    sinon :
        Renvoyer une erreur
```

## 9) Fonction d'acquisition

La fonction d'acquisition va, comme son nom l'indique, acquérir les données depuis les différents capteurs (gps, luminosité, température, pression et humidité) avec l'intervalle définie par la configuration. Elle va traiter les erreurs si il y en a et retourner les données avec l'heure, les minutes et la seconde de l'enregistrement.

```
fonction acquerirDonnees()
    Déclarer variable debutTemps = temps actuel
    Déclarer variable donnees
    donnees += heures, minutes, secondes
    Si modeActuel = "MODE_ECONOMIQUE" :
        Déclarer variable intervalleLog = valeur d'intervalle de la configuration * 2
    Sinon :
        Déclarer variable intervalleLog = valeur d'intervalle de la configuration
    Si temps actuel - debutTemps >= intervalleLog :
        Déclarer variable donnees = requete API
        Si erreur d'accès aux donnees des capteurs
            donnees += "NA"
            Retourner "ERREUR_CAPTEURS"
        Sinon :
            donnees += luminosite, temperature, humidite, pression
        Si modeActuel != "MODE_ECONOMIQUE" :
            Si erreur d'accès aux donnees du GPS:
                donnees += "NA"
                Retourner "ERREUR_GPS"
            Sinon :
                donnees += latitude, longitude
        Si donnees cohérentes :
            Retourner donnees
        Sinon :
            Retourner "ERREUR_DONNEES"
```

### 10 ) Fonction de sauvegarde

Enfin, la fonction de sauvegarde va permettre d'enregistrer les données récupérées par les capteurs sur la carte SD, avec toujours la gestion des erreurs, mais aussi la gestion des fichiers avec un système de révisions.

```
fonction sauvegarderDonnees(donnees):
    Déclarer variable fichierCourant = anneeMoisJour + "_0.LOG"

    Si fichierCourant n'existe pas:
        Créer un fichier fichierCourant

    Ouvrir fichierCourant en mode écriture

    Si taille(fichierCourant) + taille(donnees) > config.Filemaxsize:
        Fermer fichierCourant
        Si taille(donnees) + taille(carteSD) > 8go:
            Retourner erreur "Carte SD pleine":
        Créer un nouveau fichier anneeMoisJour + "_1.LOG"
        Copier fichierCourant vers le nouveau fichier
        Effacer fichierCourant

    Ecrire donnees dans fichierCourant
    Si erreur lors de l'écriture:
        Retourner erreur "Erreur lecture/écriture carte SD"

    Fermer fichierCourant
```

# Conclusion :

Grâce à la création de notre pseudo-code, nous avons maintenant une vision détaillée sur l'ensemble des fonctions, sur les variables ainsi que sur les différentes conditions et boucles que nous allons utiliser pour implémenter notre programme final en C. Il permet donc d'accélérer notre processus de développement, mais surtout d'avoir une vision claire sur ce que nous devons réaliser dans le prochain livrable.